

Python & OpenCV Task

Drone Systems — Full Solution Submission

This document contains all 20 task questions, the corresponding Python/OpenCV solution code, captured program output, and screenshots of the OpenCV detection results. The full source code and a setup README are also provided in the accompanying GitHub repository.

Q1) Drone Movement Selector

Take the index of an option (1=Roll, 2=Pitch, 3=Yaw) as input and print the corresponding drone movement instruction.

Code:

```
"""
Q1) Drone Movement Selector
-----Take the number (index of the option) as
input from the user and perform the corresponding drone movement (Roll
/ Pitch / Yaw). """

# Step 1: Display the available movement options to the
user print("Drone Movement Options:") print("1. Roll")
print("2. Pitch") print("3. Yaw")

# Step 2: Take the option number as input from the user # int()
converts the string input into an integer for comparison choice =
int(input("Enter the index of the movement option (1/2/3): "))

# Step 3: Use if-elif-else to decide which movement to perform based on
choice if choice == 1:      # Roll -> rotation about the front-to-back
(longitudinal) axis      # To roll, the drone slows down motors on one side
and speeds up the other
    print("Output: Slow down left motors for left roll OR slow down right motors for right roll")

elif choice == 2:      # Pitch -> rotation about the side-to-side (lateral) axis      # To
pitch, the drone slows down front motors (move forward) or back motors (move backward)
    print("Output: Slow down front motors to pitch forward OR slow down rear motors to pitch
backward")

elif choice == 3:      # Yaw -> rotation about the vertical axis (drone
spins/rotates in place)      # To yaw, the drone changes the speed of diagonally
opposite motor pairs      print("Output: Increase speed of clockwise motors to yaw
left OR increase speed of      counter-clockwise motors to yaw right")

else:      # Handles any input that is not 1,
2, or 3
    print("Invalid choice! Please enter 1 (Roll), 2 (Pitch), or 3 (Yaw).")
```

Output:

```
Drone Movement Options:
1. Roll
2. Pitch
3. Yaw
Enter the index of the movement option (1/2/3): 2
Output: Slow down front motors to pitch forward OR slow down rear motors
to pitch backward
```

Q2) Drone Weight Checker

Take body weight and payload weight as input (with a choice of kg or grams for extra creativity), compute the total, and report whether it exceeds 2 kg.

Code:

```
"""
Q2) Drone Weight Checker
-----Take the drone's body weight and payload
weight as input, output the total weight, and print whether the
total is greater than 2 kg.

Creative twist: the user is asked to choose the unit (kg or grams)
for each input. If grams are chosen, the value is converted to kg
before adding, since 1 kg = 1000 g. """

def get_weight_in_kg(label):    """    Helper function that asks the
user for a weight value and the unit    it is given in, then returns
the equivalent weight in kilograms.    'label' is just a description
(e.g. 'Body weight') used in the prompt.
"""    # Ask which unit the user wants to enter the weight in
unit = input(f"Enter unit for {label} (kg/g): ").strip().lower()

    # Ask for the numeric value; float() allows decimal weights
value = float(input(f"Enter {label} (in {unit}): "))

    # Convert to kg depending on the chosen unit
if unit == "g":
    return value / 1000    # 1000 grams = 1 kg
else:
    return value    # already in kg (default
assumption)

# Step 1: Get body weight in kg (handles both kg and gram input)
body_weight = get_weight_in_kg("Body weight")

# Step 2: Get payload weight in kg (handles both kg and gram input)
payload_weight = get_weight_in_kg("Payload weight")

# Step 3: Calculate total weight by simple addition
total_weight = body_weight + payload_weight

# Step 4: Print the total weight, rounded to 2 decimal places for neatness
print(f"\nOutput: {round(total_weight, 2)} kg")

# Step 5: Compare total weight against the 2 kg threshold and print result
if total_weight > 2:
    print("Total weight is greater than 2 kg")
elif total_weight == 2:
    print("Total weight is exactly 2 kg")
else:
    print("Total weight is less
than 2 kg")
```

Output:

```
Enter unit for Body weight (kg/g): kg
Enter Body weight (in kg): 5
Enter unit for Payload weight (kg/g): kg
Enter Payload weight (in kg): 3

Output: 8.0 kg
Total weight is greater than 2 kg
```

Q3) Drone Specifications Analyzer

Given fixed drone specs (frame, battery, motors, GPS), calculate the maximum payload, print each variable's type, check if a 1.8 kg camera can be carried, and convert the takeoff weight to grams as an int.

Code:

```
"""
Q3) Drone Specifications Analyzer
-----Given the drone specs below, calculate
max payload, print variable types, check if it can carry a camera, and
convert takeoff weight to grams (int). """

# ----- Given drone specifications ----max_takeoff_weight = 4.5      # kg, float ->
# heaviest the drone can weigh fully loaded frame_weight = 1.2        # kg, float
# -> weight of the bare frame battery_weight = 0.8                    # kg, float -> weight of
# the battery num_propellers = 4                                     # int      -> number of propellers
# fitted motor_weight = 0.075                                         # kg per motor, float is_gps_enabled = True
# bool      -> whether GPS module is fitted gps_module_weight = 0.05  # kg,
# float -> weight of the GPS module

# -----
# 1. Calculate the maximum payload the drone can carry
# -----#
Total weight of all motors = weight per motor * number of propellers/motors
total_motor_weight = motor_weight * num_propellers

# Start with the structural (non-payload) weight: frame + battery + motors
structural_weight = frame_weight + battery_weight + total_motor_weight

# Add GPS module weight only if GPS is actually
# enabled/fitted if is_gps_enabled:      structural_weight +=
# gps_module_weight

# Max payload = max takeoff weight - everything that isn't payload
max_payload = max_takeoff_weight - structural_weight

print(f"Total motor weight: {total_motor_weight} kg") print(f"Total structural weight
(frame+battery+motors+gps): {round(structural_weight, 3)} kg") print(f"Maximum payload the
drone can carry: {round(max_payload, 3)} kg\n")

# -----
# 2. Print the type() of each variable # -----
-----print("Variable types:")
print(f"max_takeoff_weight -> {type(max_takeoff_weight)}")
print(f"frame_weight      -> {type(frame_weight)}")
print(f"battery_weight    -> {type(battery_weight)}")
print(f"num_propellers    -> {type(num_propellers)}")
print(f"motor_weight      -> {type(motor_weight)}")
print(f"is_gps_enabled    -> {type(is_gps_enabled)}")
print(f"gps_module_weight  -> {type(gps_module_weight)}\n")

# -----
# 3. Check whether payload is enough to carry a 1.8 kg camera # -----
-----camera_weight = 1.8
# kg can_carry_camera = max_payload >= camera_weight # boolean comparison
print(f"Can the drone carry a {camera_weight} kg camera?
{can_carry_camera}\n")

# -----
# 4. Convert max_takeoff_weight to grams and store as an int
# -----# 1
kg = 1000 g; int() truncates any decimal part after conversion
max_takeoff_weight_grams = int(max_takeoff_weight * 1000) print(f"Max
takeoff weight in grams (int): {max_takeoff_weight_grams} -> type:
{type(max_takeoff_weight_grams)}")
```

Output:

```
Total motor weight: 0.3 kg
Total structural weight (frame+battery+motors+gps): 2.35 kg
Maximum payload the drone can carry: 2.15 kg

Variable types:
max_takeoff_weight -> <class 'float'>
frame_weight       -> <class 'float'>
battery_weight     -> <class 'float'>
num_propellers     -> <class 'int'>
motor_weight       -> <class 'float'>
is_gps_enabled     -> <class 'bool'>
gps_module_weight  -> <class 'float'>

Can the drone carry a 1.8 kg camera? True

Max takeoff weight in grams (int): 4500 -> type: <class 'int'>
```

Q4) Drone Telemetry Log Parser

Parse a pipe-delimited telemetry string into Drone ID, Altitude, Speed, Battery and GPS fields, check the status, replace a value, and print a formatted summary.

Code:

```
"""
Q4) Drone Telemetry Log Parser
-----Parse a pipe-separated telemetry
string, extract individual fields, check status, replace a value,
and print a formatted summary. """

telemetry =
"DRONE_ID:PX4_001|ALT:120.5m|SPEED:18.3kmph|BATT:67%|STATUS:HOVERING|GPS:23.0225N,72.5714E"

# -----
# 1. Extract Drone ID, Altitude (float), Speed (float), Battery (int), GPS
# -----# Split the whole
string on "|" to get each "KEY:VALUE" field as a list fields = telemetry.split("|") #
fields = ['DRONE_ID:PX4_001', 'ALT:120.5m', 'SPEED:18.3kmph', 'BATT:67%',
'STATUS:HOVERING',
'GPS:23.0225N,72.5714E']

# Build a dictionary {KEY: VALUE} by splitting each field on the first
":" data = {} for field in fields:     key, value = field.split(":", 1)
# split only on the first colon     data[key] = value

# Drone ID is already a plain string, no cleanup needed
drone_id = data["DRONE_ID"]

# Altitude has a trailing "m" -> strip it and convert to float
altitude = float(data["ALT"].rstrip("m"))

# Speed has a trailing "kmph" -> strip it and convert to float
speed = float(data["SPEED"].rstrip("kmph"))

# Battery has a trailing "%" -> strip it and convert to int
battery = int(data["BATT"].rstrip("%"))

# GPS coordinates are already in "lat,long" form, e.g. "23.0225N,72.5714E"
gps_coords = data["GPS"]

status = data["STATUS"]

print("Extracted Telemetry Data:") print(f"Drone ID :
{drone_id}") print(f"Altitude : {altitude} (type:
{type(altitude)})") print(f"Speed      : {speed} (type:
{type(speed)})") print(f"Battery   : {battery} (type:
{type(battery)})") print(f"GPS       : {gps_coords}\n")

# -----
# 2. Check if status contains "HOVER" (case-insensitive)
# -----
-# .upper() normalises both sides so the comparison ignores letter
case is_hovering = "HOVER" in status.upper() print(f"Is the drone
hovering? {is_hovering}\n")

# -----
# 3. Replace "HOVERING" with "RETURNING_HOME"
# -----#
.replace() works on the original telemetry string and returns a NEW string
updated_telemetry = telemetry.replace("HOVERING", "RETURNING_HOME")
print("Updated telemetry string:") print(updated_telemetry) print()

# -----
# 4. Print a formatted summary using f-strings # -----
-----summary = f"[{drone_id}] Alt: {altitude}m | Batt: {battery}% | Coords:
{gps_coords.replace(',',' ', 1)}" print("Formatted Summary:") print(summary)
```

Output:

Extracted Telemetry Data:

Drone ID : PX4_001

Altitude : 120.5 (type: <class 'float'>)

Speed : 18.3 (type: <class 'float'>)

Battery : 67 (type: <class 'int'>)

GPS : 23.0225N,72.5714E

Is the drone hovering? True

Updated telemetry string:

DRONE_ID:PX4_001|ALT:120.5m|SPEED:18.3kmph|BATT:67%|STATUS:RETURNING_HOME|GPS:23.0225N,72.5714E

Formatted Summary:

[PX4_001] Alt: 120.5m | Batt: 67% | Coords: 23.0225N, 72.5714E

Q5) Flight Path Analyser

Given a list of altitude readings, find the max altitude (and when it occurred), the average altitude, the climb-rate list, the steepest climb/descent, and the flight path with zero-altitude entries removed.

Code:

```
"""
Q5) Flight Path Analyser
-----Analyse a list of altitude readings:
find max altitude, average, climb rates, steepest climb/descent,
and trim zero-altitude entries. """

altitudes = [0, 15, 42, 78, 120, 118, 115, 50, 20, 0]

# -----
# 1. Find the maximum altitude and the index ("second") it occurred at # -----
# -----max_altitude = max(altitudes) #
highest value in the list max_index = altitudes.index(max_altitude) # first index where it
occurs (acts as the "second") print(f"Maximum altitude: {max_altitude}m, occurred at
second/index: {max_index}")

# -----
# 2. Calculate the average altitude (rounded to 2 decimal places) # --
# -----
average_altitude = round(sum(altitudes) / len(altitudes), 2)
print(f"Average altitude: {average_altitude}m")

# -----
# 3. Create climb_rates list: climb_rates[i] = altitudes[i+1] - altitudes[i]
# -----#
range(len(altitudes) - 1) stops one short so we never read past the list end
climb_rates = [altitudes[i + 1] - altitudes[i] for i in range(len(altitudes) -
1)] print(f"Climb rates between consecutive readings: {climb_rates}")

# -----
# 4. Find the steepest climb (max positive rate) and steepest descent (min/most negative
rate) # -----steepest_climb
= max(climb_rates) # largest positive change = fastest ascent steepest_descent =
min(climb_rates) # most negative change = fastest descent print(f"Steepest climb:
{steepest_climb}m/step") print(f"Steepest descent: {steepest_descent}m/step")

# -----
# 5. Remove all zero-altitude entries and print the trimmed path
# -----
# List comprehension keeps only values that are NOT equal to 0
trimmed_path = [alt for alt in altitudes if alt != 0]
print(f"Trimmed flight path (zero-altitude entries removed): {trimmed_path}")
```

Output:

```
Maximum altitude: 120m, occurred at second/index: 4
Average altitude: 55.8m
Climb rates between consecutive readings: [15, 27, 36, 42, -2, -3, -65, -30, -20]
Steepest climb: 42m/step
Steepest descent: -65m/step
Trimmed flight path (zero-altitude entries removed): [15, 42, 78, 120, 118, 115, 50, 20]
```


Q6) Waypoint Mission Planner

Work with a tuple of waypoint tuples: count them, unpack and print each one, find the highest-altitude waypoint, check membership, and demonstrate that tuples are immutable by catching the error raised on a modification attempt.

Code:

```
"""
Q6) Waypoint Mission Planner
-----Work with a tuple of waypoint tuples:
counting, unpacking, finding the highest altitude waypoint,
membership check, and demonstrating tuple immutability. """

waypoints = (      ("WP1",
23.0225, 72.5714, 50),
              ("WP2", 23.0312, 72.5801, 80),
              ("WP3", 23.0401, 72.5900, 100),
              ("WP4", 23.0225, 72.5714, 0),
            )

# -----
# 1. Print the total number of waypoints # -----
-----print(f"Total number of
waypoints: {len(waypoints)}\n")

# -----
# 2. Use tuple unpacking to print each waypoint's details in a formatted line
# -----
print("Waypoint details:") for wp in waypoints:      # Unpack the 4 elements
of each waypoint tuple directly into named variables    name, lat, lon, alt
= wp      print(f"{name}: Latitude={lat}, Longitude={lon}, Altitude={alt}m")
print()

# -----
# 3. Find the waypoint with the highest altitude using a loop
# -----
# Start by assuming the first waypoint has the highest altitude
highest_wp = waypoints[0]

for wp in waypoints:      # wp[3] is the altitude
element of the waypoint tuple    if wp[3] >
highest_wp[3]:      highest_wp = wp  # found a new
highest -> update

print(f"Waypoint with highest altitude: {highest_wp[0]} at {highest_wp[3]}m\n")

# -----
# 4. Check if ("WP2", 23.0312, 72.5801, 80) exists in waypoints # ----
-----
target_wp = ("WP2", 23.0312, 72.5801, 80)
exists = target_wp in waypoints  # 'in' checks membership inside the tuple of tuples
print(f"Does {target_wp} exist in waypoints? {exists}\n")

# -----
# 5. Try to modify WP1's altitude to 60 -> catch the error (tuples are
immutable) # -----
----try:      # Tuples (and the inner waypoint tuples) do NOT support item
assignment    waypoints[0][3] = 60 except TypeError:      # TypeError is
raised because tuples don't allow modification after creation
print("Waypoint data is immutable - mission integrity protected!")
```

Output:

Total number of waypoints: 4

Waypoint details:

WP1: Latitude=23.0225, Longitude=72.5714, Altitude=50m

WP2: Latitude=23.0312, Longitude=72.5801, Altitude=80m

WP3: Latitude=23.0401, Longitude=72.59, Altitude=100m

WP4: Latitude=23.0225, Longitude=72.5714, Altitude=0m

Waypoint with highest altitude: WP3 at 100m

Does ('WP2', 23.0312, 72.5801, 80) exist in waypoints? True

Waypoint data is immutable – mission integrity protected!

Q7) Fleet Health Monitor

Manage a dictionary representing a drone fleet: add a drone, remove one with `.pop()`, list active drones, find the lowest battery among airborne drones, bulk-update low-battery drones to a critical status, and print a summary.

Code:

```
"""
Q7) Fleet Health Monitor
-----Manage a dictionary-of-dictionaries
representing a drone fleet: add, remove, filter, find minimum,
bulk-update, and summarise. """

fleet = {
    "PX4_001": {"battery": 87, "altitude": 120, "status": "active",
    "payload_kg": 1.2},
    "PX4_002": {"battery": 23, "altitude": 0, "status": "grounded", "payload_kg": 0},
    "PX4_003": {"battery": 56, "altitude": 85, "status": "active", "payload_kg": 0.8},
    "PX4_004": {"battery": 11, "altitude": 30, "status": "returning", "payload_kg": 0.5},
}

# -----
# 1. Add "PX4_005" with battery=95, altitude=0, status="standby", payload_kg=0 # ---
-----fleet["PX4_005"] =
{"battery": 95, "altitude": 0, "status": "standby", "payload_kg": 0} print("After
adding PX4_005:") print(fleet, "\n")

# -----
# 2. Remove "PX4_002" using .pop() and print its details before removal
# -----
-# .pop(key) removes the key AND returns its value in one step
removed_drone = fleet.pop("PX4_002") print(f"Removed PX4_002, its
details were: {removed_drone}\n")

# -----
# 3. Print all active drones and their battery levels # -----
-----print("Active
drones and battery levels:") for drone_id, info in fleet.items():
if info["status"] == "active":
    print(f"{drone_id}: {info['battery']}%")
print()

# -----
# 4. Find the drone with the lowest battery among drones currently in the air (altitude >
0) # -----airborne_drones
= {d_id: info for d_id, info in fleet.items() if info["altitude"] > 0}

if airborne_drones:    # min() with a key function finds the dict entry with the smallest
battery value    lowest_battery_id = min(airborne_drones, key=lambda d_id:
airborne_drones[d_id]["battery"])    print(f"Lowest battery among airborne drones:
{lowest_battery_id} "    f"({airborne_drones[lowest_battery_id]['battery']}%)\n")
else:    print("No drones currently airborne.\n")

# -----
# 5. Update every drone with battery < 30 to status =
"critical_low_battery" # -----
-----for drone_id, info in fleet.items():    if
info["battery"] < 30:        info["status"] = "critical_low_battery"

print("Fleet after critical battery status update:")
print(fleet, "\n")

# -----
# 6. Print a final formatted fleet summary # -----
-----print("==== FINAL FLEET
SUMMARY =====") for drone_id, info in fleet.items():
    print(f"{drone_id} | Battery: {info['battery']}% | Altitude: {info['altitude']}m | "
f"Status: {info['status']} | Payload: {info['payload_kg']}kg")
```

Output:

```
After adding PX4_005:
{'PX4_001': {'battery': 87, 'altitude': 120, 'status': 'active', 'payload_kg': 1.2}, 'PX4_002': {'battery': 23, 'altitude': 0, 'status': 'grounded', 'payload_kg': 0}, 'PX4_003': {'battery': 56, 'altitude': 85, 'status': 'active', 'payload_kg': 0.8}, 'PX4_004': {'battery': 11, 'altitude': 30, 'status': 'returning', 'payload_kg': 0.5}, 'PX4_005': {'battery': 95, 'altitude': 0, 'status': 'standby', 'payload_kg': 0}}

Removed PX4_002, its details were: {'battery': 23, 'altitude': 0, 'status': 'grounded', 'payload_kg': 0}

Active drones and battery levels:
PX4_001: 87%
PX4_003: 56%

Lowest battery among airborne drones: PX4_004 (11%)

Fleet after critical battery status update:
{'PX4_001': {'battery': 87, 'altitude': 120, 'status': 'active', 'payload_kg': 1.2}, 'PX4_003': {'battery': 56, 'altitude': 85, 'status': 'active', 'payload_kg': 0.8}, 'PX4_004': {'battery': 11, 'altitude': 30, 'status': 'critical_low_battery', 'payload_kg': 0.5}, 'PX4_005': {'battery': 95, 'altitude': 0, 'status': 'standby', 'payload_kg': 0}}

===== FINAL FLEET SUMMARY =====
PX4_001 | Battery: 87% | Altitude: 120m | Status: active | Payload: 1.2kg
PX4_003 | Battery: 56% | Altitude: 85m | Status: active | Payload: 0.8kg
PX4_004 | Battery: 11% | Altitude: 30m | Status: critical_low_battery | Payload: 0.5kg
PX4_005 | Battery: 95% | Altitude: 0m | Status: standby | Payload: 0kg
```

Q8) Restricted Airspace Conflict Checker

Use set operations (intersection, difference, symmetric difference, subset check, union) to compare planned, restricted, and cleared airspace zones.

Code:

```
"""
Q8) Restricted Airspace Conflict Checker
-----Use set operations to
check planned flight zones against restricted and cleared zones. """

planned_zones = {"Z1", "Z2", "Z3", "Z5", "Z7"}
restricted_zones = {"Z3", "Z5", "Z6", "Z8"}
cleared_zones = {"Z1", "Z2", "Z3", "Z4", "Z5", "Z6", "Z7", "Z8"}

# -----
# 1. Find which planned zones are restricted (intersection)
# -----# '&'
(or .intersection()) returns elements common to BOTH sets
conflicting_zones = planned_zones & restricted_zones
print(f"Planned zones that are restricted (conflicts): {conflicting_zones}\n")

# -----
# 2. Find zones that are planned but NOT restricted (safe zones)
# -----# '-'
(or .difference()) returns elements in the first set but not the second
safe_zones = planned_zones - restricted_zones
print(f"Safe planned zones (not restricted): {safe_zones}\n")

# -----
# 3. Zones in either planned or restricted, but NOT both (symmetric difference)
# -----# '^'
(or .symmetric_difference()) returns elements in exactly one of the two sets
sym_diff_zones = planned_zones ^ restricted_zones
print(f"Zones in planned OR restricted but not both: {sym_diff_zones}\n")

# -----
# 4. Check if planned_zones is a subset of cleared_zones # -----
-----is_subset =
planned_zones.issubset(cleared_zones)
if is_subset:
    print(f"All planned zones ({planned_zones}) are cleared for flight. Mission approved.\n")
else:
    print(f"WARNING: Not all planned zones are cleared! Mission needs review.\n")

# -----
# 5. Add "Z9" to planned_zones and remove "Z7" (added by mistake) # -----
-----planned_zones.add("Z9") #
.add() inserts a single new element into the set
planned_zones.discard("Z7") #
.discard() removes an element safely (no error if missing)
print(f"Updated planned zones: {planned_zones}\n")

# -----
# 6. Print the count of unique zones across all three sets combined
# -----
# '|' (or .union()) combines all sets, automatically removing
duplicates
all_unique_zones = planned_zones | restricted_zones | cleared_zones
print(f"All unique zones combined: {all_unique_zones}")
print(f"Count of unique zones across all three sets: {len(all_unique_zones)}")
```

Output:

Planned zones that are restricted (conflicts): {'Z5', 'Z3'}

Safe planned zones (not restricted): {'Z2', 'Z1', 'Z7'}

Zones in planned OR restricted but not both: {'Z1', 'Z7', 'Z6', 'Z2', 'Z8'}

All planned zones ({'Z5', 'Z3', 'Z7', 'Z2', 'Z1'}) are cleared for flight. Mission approved.

Updated planned zones: {'Z9', 'Z5', 'Z3', 'Z2', 'Z1'}

All unique zones combined: {'Z8', 'Z4', 'Z9', 'Z5', 'Z3', 'Z7', 'Z6', 'Z2', 'Z1'}

Count of unique zones across all three sets: 9

Q9) Auto Return-to-Home System

Independently check four RTH trigger conditions (low battery, low signal, high wind, obstacle detected) and simulate a stepped 15m-per-step descent that drains 1% battery per step using a while loop.

Code:

```
"""
Q9) Auto Return-to-Home (RTH) System
-----Check independent safety
trigger rules and simulate a stepped descent that drains battery as
the drone descends. """

drone_state = {
    "battery": 18,
    "altitude": 95,
    "signal_strength": 40, # percent
    "distance_from_home": 850, # metres
    "wind_speed": 38, # km/h
    "obstacle_detected": True
}

# -----
# 1. Check ALL RTH trigger rules INDEPENDENTLY (each its own 'if', not
elif) # so that more than one warning can fire at the same time. # ----
-----

triggered_any = False # tracks whether at least one rule fired

if drone_state["battery"] < 20:
    print("CRITICAL: RTH triggered - Low Battery")
    triggered_any = True

if drone_state["signal_strength"] < 30:
    print("WARNING: RTH triggered - Signal Lost")
    triggered_any = True

if drone_state["wind_speed"] > 35:
    print("WARNING: RTH triggered - High Wind")
    triggered_any = True

if drone_state["obstacle_detected"] is True:
    print("CAUTION: Obstacle detected - Rerouting")
    triggered_any = True

# If none of the rules above triggered anything, everything is
fine if not triggered_any: print("All systems nominal")

print() # blank line separator before the descent simulation

# -----
# 2 & 3. While loop: simulate descent from current altitude to 0 in steps
# of 15m, draining 1% battery per 15m descended, printing each
step. # -----
---current_altitude = drone_state["altitude"] current_battery =
drone_state["battery"]

print("Starting descent simulation...") while current_altitude > 0:
    # Decide how much to descend this step: 15m, but not below 0 step =
min(15, current_altitude) current_altitude -= step # descend by
the step amount current_battery -= 1 # drain 1% battery for
this descent step

    # Guard so displayed battery never goes negative even in edge cases
display_battery = max(current_battery, 0)

    print(f"Altitude: {current_altitude}m | Battery: {display_battery}%")

print("Descent complete. Drone has landed.")
```

Output:

CRITICAL: RTH triggered – Low Battery
WARNING: RTH triggered – High Wind
CAUTION: Obstacle detected – Rerouting

Starting descent simulation...

Altitude: 80m | Battery: 17%

Altitude: 65m | Battery: 16%

Altitude: 50m | Battery: 15%

Altitude: 35m | Battery: 14%

Altitude: 20m | Battery: 13%

Altitude: 5m | Battery: 12%

Altitude: 0m | Battery: 11%

Descent complete. Drone has landed.

Q10) Choosing the Right Python Data Structure

Justify which data structure (list, tuple, or set) fits each of: A) a live sensor stream, B) a GPS home-point set once, and C) a set of unique error codes — in 200-300 words.

A) Live sensor stream (altitude, speed, battery — captured every 100 ms) A **list** is the best fit here. Lists are mutable and support fast `.append()` calls (amortised $O(1)$), which matters because new readings arrive constantly and need to be added to the end of the stream without rebuilding the whole collection. Lists also give $O(1)$ random access by index, so the program can instantly pull up "the reading from 5 seconds ago" or update a specific entry. If a tuple were used instead, every new reading would force the creation of an entirely new tuple (since tuples are immutable), which is wasteful and slow at high frequency. A set would be even worse: it has no guaranteed order and no indexing at all, so "what was the altitude 2 seconds ago" becomes impossible to answer directly.

B) GPS home-point coordinates (set once before takeoff) A **tuple** is ideal here, e.g. `home_point = (23.0225, 72.5714)`. The whole point of the home point is that it must **never** change mid-flight — accidentally overwriting it could send the drone to the wrong location during an emergency Return-to-Home. Tuples are immutable, so Python itself enforces this safety: any attempt to reassign an element raises a `TypeError`. If a list were used instead, nothing would stop a stray line of code (or a bug) from quietly mutating the coordinates mid-mission, which is a serious safety risk for something this critical.

C) Unique error codes logged during flight A **set** is the right structure. Sets automatically discard duplicates, which matches the requirement that repeated error codes like "E01" appearing twice are meaningless — we only care about **which** errors occurred, not how many times. Sets also use hashing internally, giving $O(1)$ average-case membership checks, so testing `"E99" in error_set` to check for a critical error is essentially instant. Using a list instead would mean duplicate codes pile up uselessly and checking membership requires scanning the entire list ($O(n)$), which gets slower as the flight log grows.

Q11) Recursive Waypoint Distance Calculator

Using pure recursion (no loops), compute the total path distance across a list of waypoints, and separately find the longest single leg of the journey.

Code:

```
"""
Q11) Recursive Waypoint Distance Calculator
-----Pure recursion
(no loops) to compute total path distance and the longest
single leg between consecutive waypoints. """

waypoints = [(0, 0), (3, 4), (6, 8), (10, 8), (10, 0)]

def euclidean_distance(p1, p2):    """    Straight-line distance
between two (x, y) points.    Not recursive itself (it's a one-
step formula), used as a helper    by the recursive functions
below.    """    x1, y1 = p1    x2, y2 = p2    return ((x2 -
x1) ** 2 + (y2 - y1) ** 2) ** 0.5

def total_distance(waypoints, index=0):    """    Recursively
computes the total path distance across all waypoints.

    Base case: if we're at the last waypoint (or beyond), there's no
further leg to travel, so the remaining distance is 0.

    Recursive case: distance of the CURRENT leg (index -> index+1)
PLUS the total distance of the REST of the path (index+1 onward).
    """    # Base case: only one waypoint left (or none) -> nothing more
to travel    if index >= len(waypoints) - 1:        return 0

    # Distance for just this one leg (current point to next point)
current_leg = euclidean_distance(waypoints[index], waypoints[index + 1])

    # Recursively add the distance of all remaining legs
return current_leg + total_distance(waypoints, index + 1)

def find_longest_leg(waypoints, index=0, max_dist=0):    """
Recursively finds the longest single leg (segment) in the journey.

    Base case: if we're at the last waypoint (or beyond), there's no
more legs to compare, so we simply return the max_dist found so far.

    Recursive case: compute the current leg's distance, compare with
max_dist found so far, and pass the larger of the two onward to
the next recursive call.    """    # Base case: no more legs left
to check -> return what we've found    if index >= len(waypoints) -
1:        return max_dist

    # Distance of the current leg    current_leg =
euclidean_distance(waypoints[index], waypoints[index + 1])

    # Update max_dist if this leg is longer than anything found so far
new_max = max(max_dist, current_leg)

    # Recurse onward to check the rest of the legs
return find_longest_leg(waypoints, index + 1, new_max)

# -----
# Run and display results # -----
-----print(f"Waypoints: {waypoints}\n") total =
total_distance(waypoints)
print(f"Total path distance: {round(total, 3)} units")

longest_leg = find_longest_leg(waypoints)
print(f"Longest single leg distance: {round(longest_leg, 3)} units")
```

Output:

Waypoints: [(0, 0), (3, 4), (6, 8), (10, 8), (10, 0)]

Total path distance: 22.0 units

Longest single leg distance: 8.0 units

Q12) OOP Concepts in Python — The Four Pillars

Explain Object-Oriented Programming concepts in Python and discuss its four pillars.

Object-Oriented Programming (OOP) organizes code around **objects** — bundles of data (attributes) and behavior (methods) modeled on real-world entities (e.g. a `Drone` class with `battery`, `altitude` attributes and a `take_off()` method). Python supports OOP fully, and it rests on four core pillars:

- 1. Encapsulation** Encapsulation means bundling an object's data and the methods that operate on that data together inside a class, and restricting direct access to internal details. In Python, this is commonly done with a leading underscore (`_battery`) for "internal use" or double underscore (`__battery`) for name-mangled private attributes, paired with getter/setter methods or `@property` decorators. This protects an object's internal state from being changed in invalid ways from outside the class — e.g. preventing `drone.battery = -50`, which makes no physical sense.
- 2. Abstraction** Abstraction means hiding complex implementation details and exposing only the essential features a user of the class needs. For example, a `Drone` class might expose a simple `.land()` method, while internally it handles motor throttling, sensor checks, and stabilization logic that the caller never has to think about. Python supports this through abstract base classes (the `abc` module), which let you define a required interface without specifying *how* it's implemented.
- 3. Inheritance** Inheritance allows one class (the child/subclass) to reuse and extend the attributes and methods of another class (the parent/superclass). For example, a `Quadcopter` class could inherit from a general `Drone` class, automatically gaining its `take_off()` and `land()` methods while adding its own specialized behavior. This avoids rewriting common code and models natural "is-a" relationships (a `Quadcopter` *is a* `Drone`).
- 4. Polymorphism** Polymorphism means objects of different classes can be used interchangeably through a shared interface, with each class providing its own specific behavior for the same method call. For example, both `Quadcopter` and `FixedWingDrone` might implement a `move()` method, but each does it differently internally — yet calling `drone.move()` works correctly regardless of which type `drone` actually is. Python achieves this naturally through duck typing ("if it walks like a duck and quacks like a duck...") and method overriding.

Together, these four pillars let large programs (like fleet-management or flight-control software) stay organized, reusable, and easier to extend safely as requirements grow.

Q13) Drone Coordinate Mover

Take starting (x, y) coordinates, a movement direction (horizontal/vertical), and a number of steps, then print the updated coordinate after each single-unit move.

Code:

```
"""
Q13) Drone Coordinate Mover
-----Take starting (x, y) coordinates, a
movement direction, and number of steps. Print the updated
coordinate after EACH single-unit move. """

# Take both coordinates together on one line, e.g. "5,3" or "5 3"
raw_coords = input("Enter the starting coordinates as x,y: ")

# Split on comma (and strip spaces) to support "5,3" or "5,
3" x_str, y_str = raw_coords.replace(" ", "").split(",") x,
y = int(x_str), int(y_str)

# Ask for movement direction direction = input("Enter movement (horizontal
or vertical): ").strip().lower()

# Ask for number of steps to move steps =
int(input("Enter the number of steps: "))

print("Output:", end=" ")

# Move one unit at a time, printing the coordinate after each single
move for _ in range(steps):    if direction == "horizontal":
x        += 1 # horizontal movement changes the x-coordinate
elif direction == "vertical":
y        += 1 # vertical movement changes the y-coordinate
else:
    print("\nInvalid direction! Please enter 'horizontal' or 'vertical'.")
    break

    # Print the coordinate after this single-unit move (no newline, like sample output)
print(f"({x},{y})", end=" ")

print() # final newline for clean formatting
```

Output:

```
Enter the starting coordinates as x,y: 5,8
Enter movement (horizontal or vertical): horizontal
Enter the number of steps: 5
Output: (6,8) (7,8) (8,8) (9,8) (10,8)
```

Q14) Drone Login Authentication

Take a user id and password, convert both to lowercase, and compare with predefined credentials to print a success or failure message.

Code:

```
"""
Q14) Drone Login Authentication
-----Take a user id and password,
convert both to lowercase, and compare against predefined
credentials. """

# Predefined (correct) credentials, stored already in lowercase
PREDEFINED_USER_ID = "admin"
PREDEFINED_PASSWORD = "drone123"

# Take user input for id and password
entered_user_id = input("Enter user id: ")
entered_password = input("Enter password: ")

# Convert both entered values to lowercase before comparing #
(this makes the login case-insensitive, e.g. "Admin" == "admin")
entered_user_id_lower = entered_user_id.lower()
entered_password_lower = entered_password.lower()

# Compare with the predefined credentials if entered_user_id_lower == PREDEFINED_USER_ID and
entered_password_lower == PREDEFINED_PASSWORD:
    print("Drone connected successfully") else:
print("Connection failed: Invalid user id or password")
```

Output:

```
Enter user id: AdMiN
Enter password: drone123
Drone connected successfully
```

Q15) Vowel Counter

Take a string from the user, print it, and count the number of vowels present.

Code:

```
"""
Q15) Vowel Counter
-----
Take a string from the user, print it, and count the number of
vowels (a, e, i, o, u – both uppercase and lowercase) present in
it. """

text = input("Enter a string: ")

vowels = "aeiouAEIOU" # all vowels, both cases

count = 0
for char in text:
    if char in vowels:
        count += 1

print(f"String entered: {text}")
print(f"Number of vowels present: {count}")
```

Output:

```
Enter a string: RAMCHandra
String entered: RAMCHandra
Number of vowels present: 3
```

Q16) Lowercase → Uppercase using ord()/chr()

Convert a single lowercase letter to uppercase using ONLY ord() and chr() (no built-in string methods like .upper()).

Code:

```
"""
Q16) Lowercase to Uppercase using ord() and chr()
-----Convert a
single lowercase letter to uppercase using ONLY ord() and chr() -
no built-in string methods like .upper() allowed.

Background (ASCII):
- Lowercase letters 'a' to 'z' occupy ASCII codes 97 to 122.
- Uppercase letters 'A' to 'Z' occupy ASCII codes 65 to 90. - The gap between a
  lowercase letter and its uppercase version is always exactly 32 (e.g.
  ord('a')=97, ord('A')=65, 97-65=32). - So subtracting 32 from a lowercase
  letter's ASCII code gives the ASCII code of its uppercase equivalent. """

letter = input("Enter the letter: ")

# Get the ASCII (numeric) code of the entered character
ascii_code = ord(letter)

# Subtract 32 to shift from lowercase range (97-122) to uppercase range (65-90)
uppercase_ascii_code = ascii_code - 32

# Convert the new ASCII code back into a character
uppercase_letter = chr(uppercase_ascii_code)

print(f"Output: {uppercase_letter}")
```

Output:

Enter the letter: a

Output: A

Q17) Drone Landing Pad Detector

Detect a white rectangular landing pad on a dark background from a drone camera frame, draw a box around it, print 'Landing', and save the annotated frame as sahi.jpg. (Since no real drone footage was supplied, a synthetic test frame was generated to verify the detection logic end-to-end — shown below.)

Code:

```
"""
Q17) Drone Landing Pad Detector
-----Detect a rectangular white
landing pad on a dark background from a drone camera feed image,
and "land" the drone on it (just print a landing message — no real
flight control). Save the annotated result as 'sahi.jpg'.

NOTE: Replace INPUT_IMAGE_PATH below with the actual path to your
drone camera frame. A synthetic test frame
('landing_pad_frame.jpg') is provided in the sample_images folder
for testing. """

import cv2 INPUT_IMAGE_PATH = "landing_pad_frame.jpg" # <-- replace with your own drone
frame if needed

# Step 1: Load the drone camera frame from disk
img = cv2.imread(INPUT_IMAGE_PATH)

if img is None:     raise FileNotFoundError(f"Could not load image at
'{INPUT_IMAGE_PATH}'. Check the path.")

# Step 2: Convert to grayscale -> easier to threshold brightness than full color
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Step 3: Threshold the grayscale image so only bright (white) pixels survive
# Pixels above 200 brightness become white (255), everything else becomes black (0)
_, thresh = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY)

# Step 4: Find the outlines (contours) of the white blobs left after thresholding
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

landing_pad_found = False
best_contour = None
best_area = 0

# Step 5: Examine each contour to find the one that looks like our landing
pad for cnt in contours:     area = cv2.contourArea(cnt)

    # Ignore tiny specks of white noise; a real landing pad will be reasonably
    large     if area < 500:         continue

    # Get the bounding rectangle of this contour to check its
    shape     x, y, w, h = cv2.boundingRect(cnt)     aspect_ratio = w
    / float(h)

    # A landing pad should be roughly rectangular (not extremely thin/long)
    if 0.3 < aspect_ratio < 3.0 and area > best_area:
        best_area = area
    best_contour = (x, y, w, h)
    landing_pad_found = True

# Step 6: Draw a bounding box around the detected landing pad (if found)
if landing_pad_found:
    x, y, w, h = best_contour     cv2.rectangle(img, (x, y), (x + w, y +
h), (0, 255, 0), 3) # green box

    # Mark the center point of the landing pad
    center_x, center_y = x + w // 2, y + h // 2
    cv2.circle(img, (center_x, center_y), 5, (0, 0, 255), -1) # red center dot

    cv2.putText(img, "LANDING PAD", (x, y - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

    # Step 7: "Land" the drone — just print the message as instructed     print(f"Landing
pad detected at center ({center_x}, {center_y}), area={best_area:.0f} px")
    print("Landing") else:
        print("No landing pad detected. Holding position.")
```

```
# Step 8: Save the annotated result image as  
required cv2.imwrite("sahi.jpg", img) print("Result  
saved as 'sahi.jpg'")
```

Output:

Landing pad detected at center (300, 200), area=19200 px
Landing
Result saved as 'sahi.jpg'

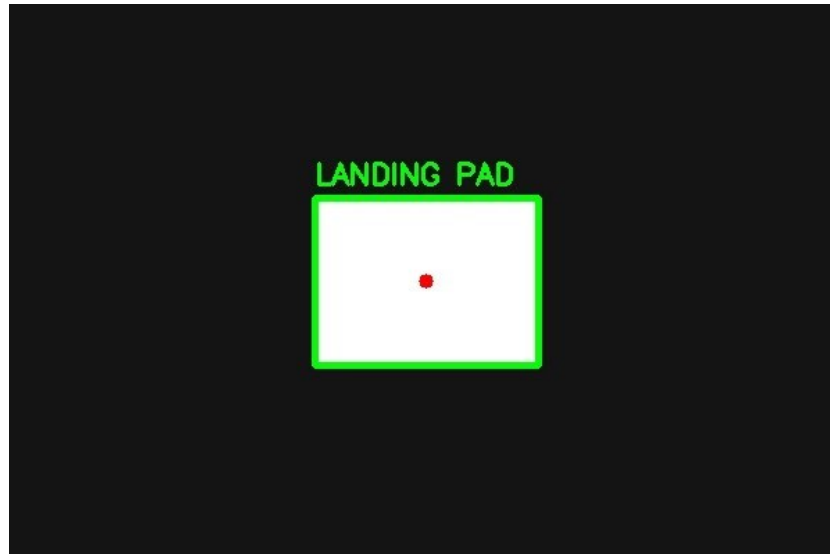


Fig 1: Landing pad correctly detected and boxed (synthetic test frame)

Q18) Drone Camera Red Object Tracker

Simulate a drone camera following a red object in live video using HSV color filtering, drawing a circle around the detected object, and overlaying 'Object detected' / 'Object lost' status text. The script runs live against cv2.VideoCapture(0); a TEST_MODE flag is included to verify the detection logic headlessly against a static red-object test image, shown below.

Code:

```
"""
Q18) Drone Camera Red Object Tracker
-----Simulate a drone's camera
following a red object (ball/marker) in live video using OpenCV's
HSV color filtering.

Run this on a machine with a webcam attached -- it opens VideoCapture(0)
and shows a live window. Press 'q' to quit.

For headless testing/screenshots (no camera available), this file
also has a TEST_MODE flag that processes a single static demo image
instead of a live feed (see sample_images/red_object_frame.jpg). """

import cv2
import numpy as np
TEST_MODE = False # set to True to run on a
static test image instead of a webcam

def detect_red_object(frame):
    """ Takes one BGR video
    frame, finds the largest red blob using HSV color filtering,
    draws a circle around it, and overlays detection status text.
    Returns the annotated frame. """
    # Step 1: Convert frame from BGR color space to HSV
    # HSV separates color (Hue) from brightness/saturation, making
    # color-based filtering far more reliable than using raw BGR values.
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # Step 2: Define the HSV range for "red".
    # Red sits at the very start AND the very end of the Hue wheel (0 and
    180), # so two ranges are needed to fully capture all shades of red.
    lower_red_1 = np.array([0, 120, 70])
    upper_red_1 = np.array([10, 255, 255])
    lower_red_2 = np.array([170, 120, 70])
    upper_red_2 = np.array([180, 255, 255])

    # Step 3: Create binary masks for each red range and combine
    them
    mask1 = cv2.inRange(hsv, lower_red_1, upper_red_1)
    mask2 = cv2.inRange(hsv, lower_red_2, upper_red_2)
    red_mask = cv2.bitwise_or(mask1, mask2)

    # Step 4: Clean up the mask a little (remove small noise
    specks)
    red_mask = cv2.erode(red_mask, None, iterations=2)
    red_mask = cv2.dilate(red_mask, None, iterations=2)

    # Step 5: Find contours (outlines) of all red blobs in the mask
    contours, _ = cv2.findContours(red_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    object_detected = False

    if contours:
        # Pick the largest red contour -> assume that's
        our tracked object
        largest_contour = max(contours,
            key=cv2.contourArea)
        area = cv2.contourArea(largest_contour)

        # Ignore tiny red specks (noise) -- require a minimum area
        if area > 300:
            # Get the smallest circle that fully
            encloses the contour
            (x, y), radius = cv2.minEnclosingCircle(largest_contour)
            center = (int(x), int(y))
            radius = int(radius)

            # Draw the tracking circle and its center point
            cv2.circle(frame, center, radius, (0, 255, 0), 2)
            cv2.circle(frame, center, 4, (255, 255, 255), -1)
            object_detected = True

    # Step 6: Overlay status text depending on whether we found the object
    if object_detected:
        cv2.putText(frame, "Object detected", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
    else:
```

```

        cv2.putText(frame, "Object lost", (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

    return frame, object_detected

if __name__ == "__main__":
    if TEST_MODE:
        # Headless single-image test (useful when
        no webcam is available)
        frame = cv2.imread("sample_images/red_object_frame.jpg")
        result, detected = detect_red_object(frame)
        cv2.imwrite("red_object_result.jpg",
result)
        print(f"Object detected: {detected}. Result saved as 'red_object_result.jpg'")
    else:
        # Live webcam mode (this is the actual drone-camera simulation)
        cap =
cv2.VideoCapture(0) # 0 = default webcam; replace with a video file path if needed

        if not cap.isOpened():
            print("Could not access the camera. Set TEST_MODE = True to test on a static image.")
        else:
            while True:
                ret, frame = cap.read()
            if not ret:
                print("Failed to grab
frame. Exiting.")
                break

            frame, _ = detect_red_object(frame)

            cv2.imshow("Drone Red Object Tracker", frame)

            # Press 'q' to quit the live feed
            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

            cap.release()
cv2.destroyAllWindows()

```

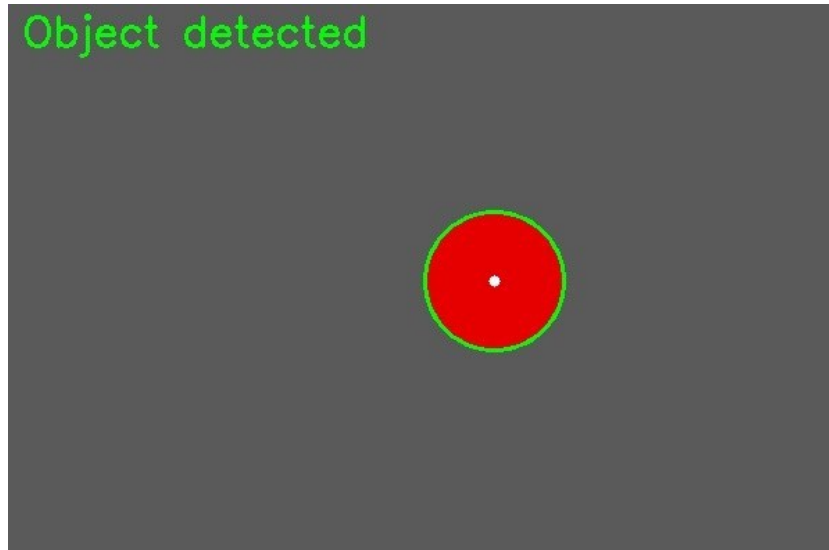


Fig 2: Red object correctly detected, circled, and labelled (synthetic test frame)

Q19) Aerial Grid View — Click to Select a Cell

Draw an 8x8 grid over an image and, on a left mouse click, print and highlight (in blue) the grid cell that was clicked, using `cv2.setMouseCallback()`. The screenshot below was produced by simulating a click at pixel (400, 150), which correctly resolves to grid cell (row=2, col=5).

Code:

```

"""
Q19) Aerial Grid View - Click to Select a Cell
-----Simulate an
aerial view from a drone: draw an 8x8 grid over an image. When the
user clicks a cell, print its (row, col) coordinates and highlight
that cell in blue.

Run this on a machine with a display -- it opens a window and
waits for mouse clicks. Press any key to close the window. """

import cv2

IMAGE_PATH = "aerial_view.jpg" # replace with your own aerial image if you like
ROWS, COLS = 8, 8

```

```

# Step 1: Load the base image img = cv2.imread(IMAGE_PATH) if img is None: raise
FileNotFoundError(f"Could not load image at '{IMAGE_PATH}'. Check the path.")

height, width = img.shape[:2]

# Step 2: Work out how big each grid cell is in
pixels cell_height = height // ROWS cell_width =
width // COLS

# Keeps track of which cell (row, col) was last clicked, so we can re-
draw # the highlight every time the display is refreshed selected_cell =
None

def draw_grid_view():
    """ Draws the 8x8 grid lines on a fresh copy of the image,
    and if a cell has been selected, highlights it with a blue rectangle. Then shows
    the result in the window. """ display_img = img.copy() # always draw on a fresh
    copy so old highlights don't stack up

    # Draw vertical grid lines for col in range(1, COLS):
    x = col * cell_width cv2.line(display_img, (x, 0), (x,
    height), (255, 255, 255), 1)

    # Draw horizontal grid lines for row in range(1, ROWS):
    y = row * cell_height cv2.line(display_img, (0, y), (width,
    y), (255, 255, 255), 1)

    # If a cell has been selected, highlight it with a blue filled-ish
    rectangle if selected_cell is not None:
        sel_row, sel_col = selected_cell top_left = (sel_col *
        cell_width, sel_row * cell_height) bottom_right = ((sel_col + 1) *
        cell_width, (sel_row + 1) * cell_height) # Blue (BGR = 255,0,0),
        thickness 3 for a visible highlight border cv2.rectangle(display_img,
        top_left, bottom_right, (255, 0, 0), 3)

    # Label the selected cell's coordinates near the top-left of the
    cell label = f"({sel_row},{sel_col})"
    cv2.putText(display_img, label, (top_left[0] + 5, top_left[1] + 20),
    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)

    cv2.imshow("Aerial Grid View", display_img)

def mouse_callback(event, x, y, flags, param):
    """ Called
    automatically by OpenCV whenever a mouse event happens on the
    window. We only care about left-button clicks.
    """
    global selected_cell

    if event == cv2.EVENT_LBUTTONDOWN: # Convert
    pixel (x, y) into a grid (row, col) index col = x
    // cell_width row = y // cell_height
    selected_cell = (row, col)

    print(f"Clicked at: x={x}, y={y} -> Grid cell: row={row}, col={col}")

    # Re-draw the grid with the new highlight applied
    draw_grid_view()

# Step 3: Show the initial grid (no cell selected yet) and register the click
handler draw_grid_view() cv2.setMouseCallback("Aerial Grid View", mouse_callback)

# Step 4: Wait for the user to press any key, then close the
window cv2.waitKey(0) cv2.destroyAllWindows()

```

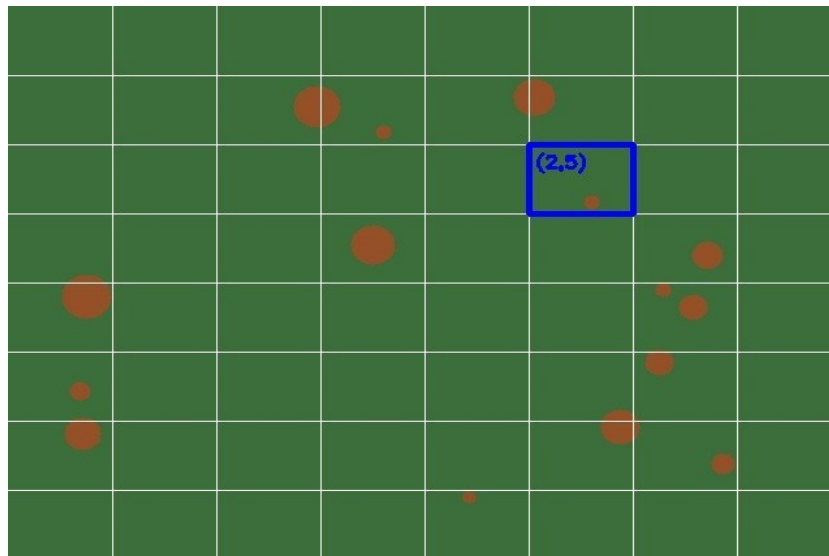


Fig 3: 8x8 grid with the clicked cell (2,5) highlighted in blue

Q20) The Logic of Colour Detection

Explain the logic behind colour detection in image processing (flowchart preferred).

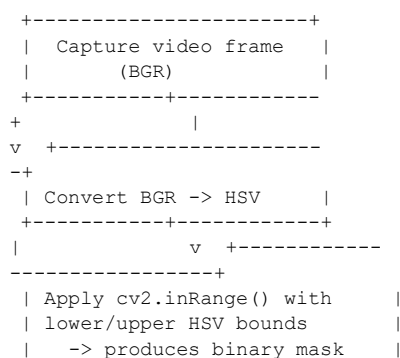
Description: Colour detection in OpenCV almost always works by converting an image from BGR (the default OpenCV colour order) into the **HSV** colour space, rather than working directly with BGR or RGB values. This is because HSV separates colour information into three independent, more intuitive components:

- **Hue (H)** — the actual colour itself (red, green, blue, etc.), represented as an angle around a colour wheel (0–180 in OpenCV's 8-bit scaling).
- **Saturation (S)** — how vivid/pure the colour is, versus washed out or gray.
- **Value (V)** — how bright or dark the colour is.

Because Hue alone captures "which colour", a target colour like red can be isolated by defining a Hue range (plus reasonable Saturation/Value minimums to avoid noise) and ignoring lighting and shadow variation almost entirely — something that's very difficult to do reliably in raw BGR, where a colour's RGB numbers change drastically under different lighting.

The overall pipeline is: 1. Capture a frame from the camera (BGR format). 2. Convert it to HSV using `cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)`. 3. Define a lower and upper HSV bound for the target colour (two ranges for red, since red wraps around both ends of the Hue circle: 0 and 180). 4. Use `cv2.inRange()` to produce a binary mask — white where the colour matches, black everywhere else. 5. Clean up the mask (erode/dilate) to remove small noise specks. 6. Find contours on the mask with `cv2.findContours()` to locate the actual blob(s) of that colour. 7. Pick the largest contour (assumed to be the target object), draw a bounding shape around it, and report "detected" or "lost" depending on whether a contour of sufficient size was found.

Flowchart:



```

+-----+-----+
+           |           v
+-----+
| Erode + Dilate mask |
| (remove noise)      |
+-----+
+           |
v +-----+
-+
| cv2.findContours() |
+-----+
|
| +-----+
| | Any large |
| | contour?  |
| +-----+
| |           |           Yes
| | No        |           v v +-----+
+-----+ +-----+
| Draw circle/ | | Print "Object |
| box, print   | | lost"         |
| "Object      | +-----+
| detected"     |
+-----+

```